Does CO_2 Predict Zone Occupancy Beyond the Weekly Schedule? Evidence from Monash Smart Infrastructure Sensor Data

MMA3001 Individual Project | Ziggy Kneebone | 33857504 | Code: https://github.com/zkne0002-art/occupancy-env-prediction

Summary

Buildings are ventilated for an assumed occupancy. This project tests whether CO_2 measured by a building sensor predicts whether a zone is occupied beyond what the hour-of-week schedule already tells us. Using about 3.5 months of 10 minute data from one working environmental sensor, adding CO_2 to a schedule only logistic regression raised held-out AUC from 0.832 to 0.905 (paired difference +0.073, 95% interval +0.034 to +0.107). A control sensor with no occupancy signal showed no gain (-0.009), so the pipeline does not manufacture improvements. A simple logistic regression matched gradient boosting and fits about four times faster. Most of the effort went into data problems: three of the five environmental sensors are dead, three quarters of environmental rows are repeated, and the sensor-to-space mapping is unreliable and had to be inferred.

1. Engineering problem

Context. Ventilation is a large share of building energy use. It is usually run to a timetable, while demand-controlled ventilation uses CO_2 as a proxy for people present. If CO_2 only reproduces the weekly timetable, it adds nothing to a schedule that is already known. The question is therefore: does CO_2 (and the other environmental variables) add predictive information about occupancy beyond the hour of the week?

Intended use. A validated, reproducible method for judging whether a building's existing environmental sensors support occupancy-aware ventilation. It is an assessment tool, not a controller, and is not suitable for safety-critical decisions.

Scope and limits. One working sensor, about 105 days of data, a zone of two inferred spaces rather than a confirmed room, a binary occupancy definition, and one chronological test window of 31 days at the end of a semester.

2. Inputs, outputs and data

The data are the Monash Smart Infrastructure sensor extracts supplied for MMA3001. Table 1 summarises what goes into and comes out of the pipeline.

Table 1. Inputs and outputs of the pipeline.

	Content	**Format, units, domain**
Input 1	Occupancy event log: 9,091,732 events, 5 spaces	CSV: floorspaceid, headcount (people, 0 or more), collecteddate (timestamp with UTC offset)
Input 2	Environmental readings: 179,447 rows, 5 sensors	CSV: sensorid, jsondata (11 variables, each with a Unix-second UTC time and units such as CO_2 in ppm, temperature in ^{o}C
Input 3	Sensor_ID_and_Locations.xlsx	Used only to report mapping coverage
Output 1	Cleaned tables	10-minute UTC grid, NaN where invalid or missing
Output 2	Mapping evidence	Residual correlation, best lag, placebo z-score and stability per space
Output 3	Model report	CV AUC, held-out metrics, paired AUC differences with 95% intervals, permutation importance, timings

Invalid or unsupported inputs. Missing required columns raise an error naming them. Unparseable JSON rows are counted and skipped. Values with an error code, or without a value, become NaN. Repeated and isolated stale readings are removed. Stale occupancy states become missing. An unknown sensor or space, no time overlap, or a single-class train or test period raises an error with a clear message.

What the data audit found (Figures 1 and 2):

- **Repeats.** 136,604 of 179,447 environmental rows (76.1%) repeat an earlier (sensor, time) pair, leaving 42,843 unique readings. The median gap between measurement and ingestion is about 136 days, consistent with stale readings being re-sent.
- **Dead sensors.** Sensors …227, …241 and …777 hold 1, 2 and 1 unique readings, re-sent 21,363, 58,908 and 10,926 times. Only …326 (29,039 unique readings, Sep 2025 to Apr 2026) and …869 (13,800, 29 Feb to 13 Jun 2024) have usable 10-minute series, with about 9% of bins empty.
- **Invalid variables.** Formaldehyde has an error code and no value in 33,638 of 42,843 unique readings (78.5%) and is excluded. PM10 is invalid in 1,693 (4.0%).
- **Events are not time.** 92.5% of occupancy events are labelled2 CurrentlyOccupied, but headcount is above zero for only about 35% of valid time (31% to 35% per space, 52% for one space). Events are logged at every headcount change, so event counts overstate occupancy; the log must be time-weighted.

- **File dates.** Files named May to Dec 2024 span Nov 2023 to Apr 2026.
- **Mapping.** The supplied spreadsheet (tab labelled ‘Incorrect Data’) matched 1 of 5 environmental sensors (...869 is in G.38, Compactus_upper) and 2 of 5 spaces (G.20, G.25); no room has both kinds of sensor. Peers report this as a known cloud-sync problem with no corrected mapping available. The mapping is therefore treated as a part of the problem.

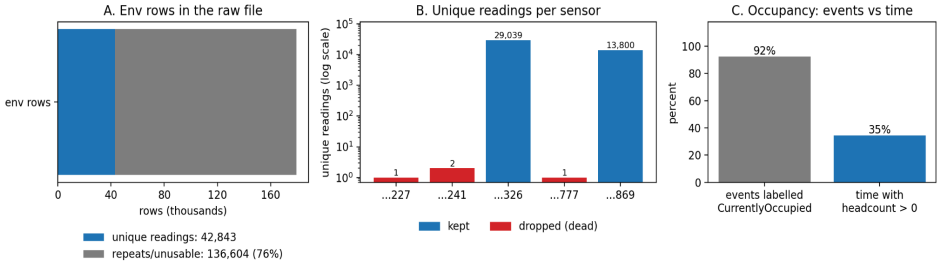


Figure 1. Data audit: (A) repeats among environmental rows; (B) unique readings per sensor on a log axis, dead sensors in red; (C) share of events labelled CurrentlyOccupied versus share of time with headcount above zero.

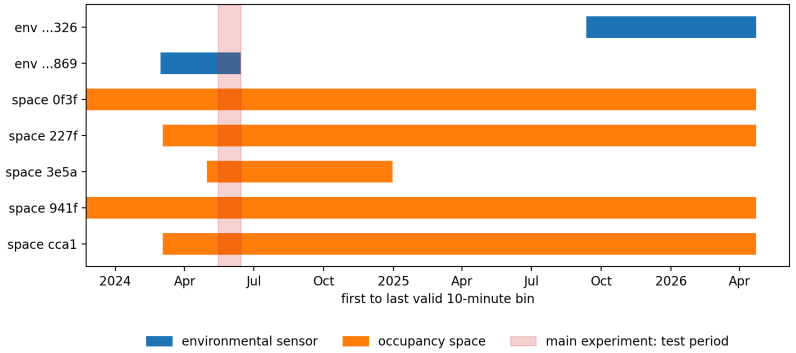


Figure 2. First to last valid bin for each sensor (blue) and space (orange); the shaded band is the test period for the main experiment.

3. Computational approach

3.1 Cleaning and regularisation

The timestamp inside the JSON (UTC) is the measurement time; createdate is only the ingest time. Exact repeats of (sensor, time) are removed. A value with an error code, or no value, becomes NaN. A reading with no neighbour within 3,600s is treated as a stale re-send, and sensors with fewer than 1,000 usable readings are dropped. Readings are averaged into 10-minute bins. The occupancy log is a step function: headcount is held between events at 1-minute resolution (state at the end of each minute) and averaged into the same bins. A non-zero headcount not refreshed for 120 minutes, or any state not refreshed for 96 hours, becomes missing, because it may be a sensor outage; a zero headcount may persist because an empty room produces no events. Bins with fewer than half valid minutes are NaN. These parameters are assumptions, set in one place in the code, and were not tuned against the test results.

3.2 Inferring the sensor-to-space mapping

Any two spaces used on weekdays correlate through the working week alone, so the evidence uses residual correlation: the mean hour-of-week profile (local time) is subtracted from the CO_2 and headcount before correlation. Each sensor is compared with every space over a common time window so correlations are comparable. Three further checks are applied: a lag scan (-3 h to +6 h; a real link peaks near 0 to 1 h), a placebo test that shifts headcount by whole weeks (this keeps the weekly rhythm but breaks the day-to-day link) and stability across the two halves of the window.

Table 2. Mapping evidence for sensor ...869, 3 Mar to 13 May 2024: residual correlation, best lag and placebo z-score per space.

Space	Residual r	Best lag (min)	Placebo z
941f0352	0.34	10	3.70
0f3f37f8	0.21	10	3.24
227f1068	0.17	70	1.78
cca1ac3e	0.10	60	1.48

Table 2 shows the training-period evidence for sensor ...869. On a fair 44-day window that also includes space 3e5a14df, three spaces are tied (r=0.59, 0.56, 0.55). I therefore model a zone of the two tied spaces with the longest coverage (0f3f37f8 and 941f0352); the zone-sum correlation over 105 days (0.47) exceeds either space alone (0.35 and 0.44). The mapping evidence was first computed on a window overlapping the test period; it was redone with data before 15 May only and picks the same two spaces (Figure 3). For sensor ...326, every residual correlation is within 0.02 of zero, placebo z-scores are -0.5 to +0.06, and best lags scatter from -160 to +120 minutes, so there is no hidden clock offset and no detectable occupancy signal. Caveat: with 44 to 71 days the placebo test has little power, so z-scores of 1.5 to 3.7 are supporting evidence only, and a strong match supports but does not prove co-location.

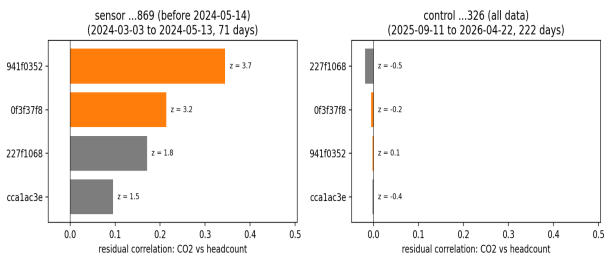


Figure 3. Residual CO_2-headcount correlation per space for sensor ...869 (training period only) and control sensor ...326; orange marks the two spaces chosen as the zone for sensor ...869 (also marked on the control panel for comparison); space 3e5a14df is left out because its data starts on 30 April, which would shorten the common window.

3.3 Prediction task, features, models and validation

Target. Occupied = zone headcount (10-minute time-weighted mean, summed over the two spaces) of at least 1 person. The zone is occupied in 47% of valid bins

(Figure 4 shows the weekly pattern). Thresholds 2 and 5 are used as sensitivity checks.

Features. Schedule: hour of day, day of week, hour of week. Environment: CO_2, temperature, humidity, PM2.5, and four engineered features (CO_2 change over 30 minutes, 1-hour mean, height above the 24-hour minimum, temperature change over 30 minutes). Every engineered feature uses only past values, which a unit test confirms by altering future readings.

Models and settings. Schedule lookup (training occupancy rate per hour of week; the baseline); logistic regression (L2, C = 1, one-hot hour of week plus standardised environmental features); histogram gradient boosting (HGB; depth 3, learning rate 0.05, 200 iterations, no early stopping). Each family is run with schedule only, schedule plus CO_2 features, and schedule plus all environmental features, plus an environment-only HGB. All models are implemented in scikit-learn.

Validation. Chronological split: training 29 Feb to 14 May 2024 (8,542 bins), a 1-day gap, test 15 May to 13 Jun (3,715 bins, 31 days). Models are compared by forward-chaining cross-validation inside the training period only, and the test set is scored once. Because 10-minute bins within a day are strongly autocorrelated, uncertainty comes from a day-block bootstrap (1,000 resamples of whole test days) of paired AUC differences. Assumptions: the occupancy sensor's headcount is accurate enough as ground truth, conditions are broadly stable across the period, and days are exchangeable.

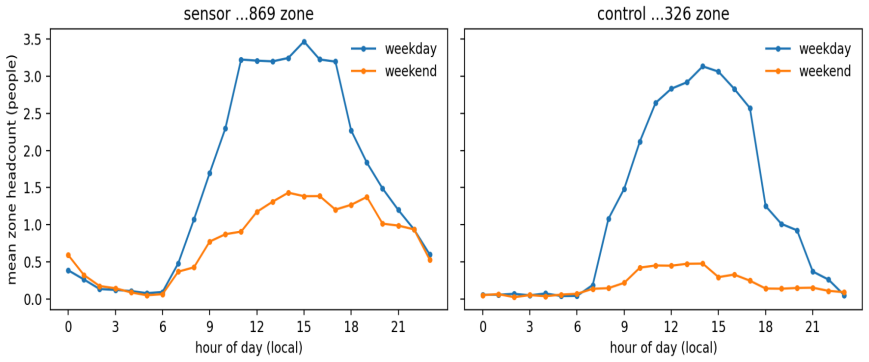


Figure 4. Mean zone headcount by local hour, weekday versus weekend, for the same two spaces in the main period (2024) and the control period (2025 to 2026). Weekend use was much higher in 2024 (about 1.4 people at midday) than in 2025 to 2026 (about 0.5), which fits the schedule baseline being stronger for the control.

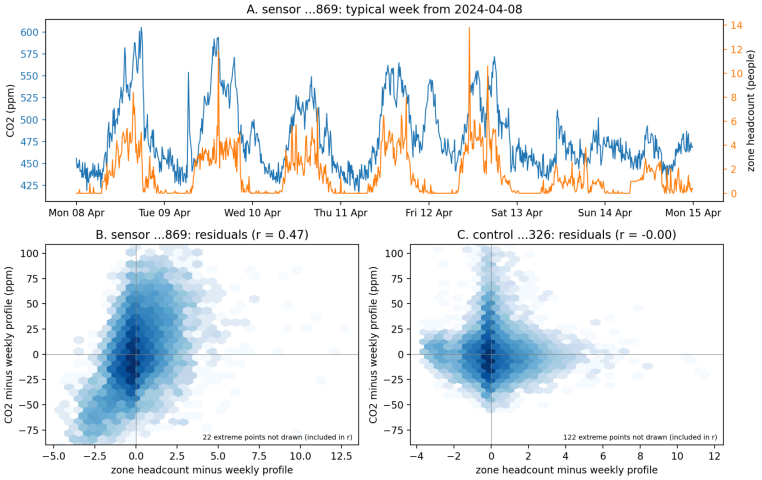


Figure 5. (A) A typical week of CO_2 and zone headcount, chosen automatically as the week with mean headcount closest to the median; (B, C) residual scatter for the main and control sensor on a shared y-axis, with correlations printed; extreme readings outside the axis range (22 for sensor ...869, 122 for the control) are counted on the figure but still included in r.

4. Alternative solutions

The schedule lookup is the baseline any environmental model must beat, because it is cheap and already available. Gradient boosting was tried as a more flexible alternative to logistic regression, and an environment-only model tests how much of the signal is just a daily clock. Each is judged on accuracy (AUC, balanced accuracy, Brier score), robustness to the data period, run time, and interpretability. Logistic regression is the most interpretable: its coefficients show how far CO_2 shifts the odds of occupancy beyond the hour of the week.

5. Validation and results

Table 3 gives the held-out results at threshold 1 and table 4 the paired differences; Figures 6 and 7 show the same results graphically.

Table 3. Held-out test results, sensor ...869, threshold 1.

Model	AUC	Balanced accuracy	Brier	Fit time (s)
Schedule lookup	0.832	0.739	0.210	0.001
Logistic, schedule	0.832	0.739	0.204	0.018
Logistic, schedule + CO_2	0.905	0.800	0.153	0.038
Logistic, schedule + all env	0.906	0.798	0.156	0.057
HGB, schedule	0.834	0.737	0.208	0.181
HGB, schedule + CO_2	0.893	0.773	0.157	0.162
HGB, schedule + all env	0.897	0.788	0.151	0.198
HGB, environmental only	0.883	0.782	0.156	0.168

Table 4. Paired test-AUC differences, A minus B, with 95% day-block bootstrap intervals.

Comparison	Threshold 1	Threshold 2	Control …326 (threshold 1)
Logistic: + CO_2 vs schedule	+0.073 [+0.034, +0.107]	+0.094 [+0.055, +0.133]	-0.009 [-0.016, -0.001]
Logistic: + all env vs schedule	+0.074 [+0.028, +0.114]	+0.103 [+0.057, +0.148]	-0.009 [-0.017, -0.002]
HGB: + CO_2 vs schedule	+0.059 [+0.015, +0.100]	+0.091 [+0.056, +0.130]	-0.007 [-0.013, -0.002]
HGB: + all env vs	+0.063 [+0.012,	+0.097 [+0.054,	-0.008 [-0.017,

schedule	+0.106]	+0.140]	+0.001]
HGB vs logistic, both + CO_2	-0.012 [-0.025, -0.000]	+0.000 [-0.009, +0.014]	+0.004 [-0.003, +0.010]
HGB schedule vs lookup	+0.002 [-0.004, +0.007]	+0.004 [-0.004, +0.011]	+0.003 [-0.001, +0.006]

Findings. (1) Environmental data adds reliable skill beyond the schedule for sensor ...869: all intervals exclude zero at thresholds 1 and 2 (Table 4, Figure 6). At threshold 5 the intervals also exclude zero, but only about 1% of test bins are positive, so that result is supplementary. (2) CO_2 carries the signal(Figure 5): CO_2-only models match all-environment models, and `co2_ppm` is the top environmental feature in permutation importance (drop in AUC 0.077, versus 0.021 for temperature). (3) Model complexity does not help (Figure 7): HGB on the schedule alone is no better than the lookup, and HGB is not better than logistic regression (with CO_2 features: -0.012 [-0.025, -0.000] at threshold 1, +0.000 [-0.009, +0.014] at threshold 2 and +0.004 [-0.003, +0.010] for the control; with all environmental features the intervals include zero in every case). (4) Negative control. On sensor ...326, with a residual CO_2-headcount correlation near zero, environmental features give no gain and, in three of four comparisons, a slight loss (about -0.007 to -0.009; the all-environment HGB interval includes zero). An environment-only HGB still scores AUC 0.726 there, because environmental variables follow a daily cycle and act as a clock; the schedule-versus-schedule-plus-environment comparison is therefore the fair test. (5) Sensitivity. With the 44-day, three-space zone (about 3,600 training bins) environmental features gave no improvement (logistic 0.822 against 0.835 for schedule only), so conclusions depend on data volume.

What validation cannot establish. That the sensor is physically in these spaces; that results hold beyond one sensor, 3.5 months and one end-of-semester test window; that the occupancy count is accurate; or anything causal. Cross-validation folds varied by 0.02 to 0.15 AUC, so it was used only for a coarse check, and the test intervals carry the conclusions.

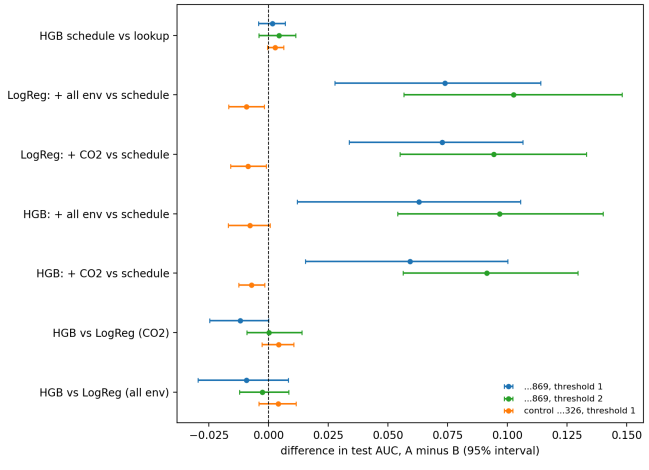


Figure 6. Paired test-AUC differences with 95% intervals for the main sensor at two thresholds and the control sensor.

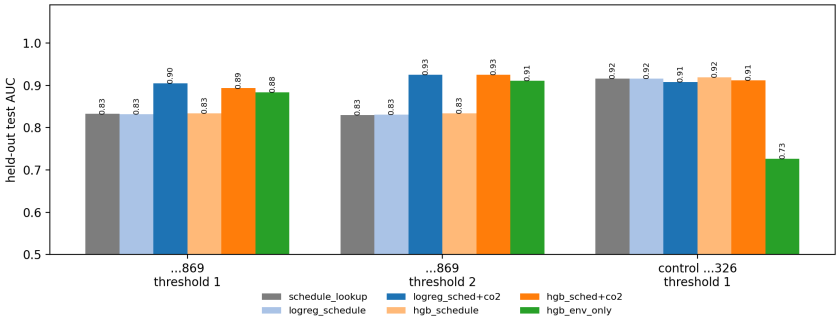


Figure 7. Held-out test AUC of the key models.

6. Optimisation and performance

Profiling (cProfile on 2 million occupancy rows and the full environmental file) located the real bottlenecks before any change was made. In the original code, timestamp parsing took 10.0s of 13.2s (76%) in cleaning the occupancy data, because pandas falls back to a slow per-row parser for offsets such as +11; reading the CSV took 2.4s. For the environmental data, JSON parsing took 7.1s of 10.2s (70%), and all 179,447 rows were parsed although 76% are discarded as repeats. (cProfile inflates Python-level loops, so speedups in Table 5 use unprofiled timings.)

Table 5. Baseline versus improved implementations (identical results confirmed on the real data in every case).

Step	Baseline	Improved	Result
Duplicate counting, 2M rows	3.35s, 361MB (set of strings)	3.02s, 79MB (vectorised hashes)	1.1x time, 4.6x less memory
Timestamp parsing, 1M rows	5.60s, 16MB	1.51s, 127MB (fixed-width parse)	3.7x faster, about 8x more memory
Environmental cleaning, full file	7.36s (parse every row)	3.98s (skip repeats before parsing)	1.85x faster; JSON calls 179,447 to 42,843

The duplicate counter is a memory improvement, not a speed improvement: the first version of its documentation claimed it was slow, and measurement showed otherwise. The faster timestamp parser trades memory for time, and parsing in chunks would cap that. After the changes, reading the CSV is the largest remaining cost (2.2s of 4.7s profiled for the environmental file); it runs once, so I stopped there.

Cost model. Analytic estimates (order of magnitude): logistic regression fit about 31 million operations at 0.25 operations per byte, HGB fit about 155 million at 1.7 operations per byte, HGB prediction 2.2 million at 0.12. Measured fit times of about 0.04 to 0.06 s (logistic regression) and 0.16 to 0.20 s (HGB) correspond to roughly 0.5 to 1 billion operations per second, far below a modern CPU's peak. With data well under a megabyte, run time is dominated by library overhead, not arithmetic or memory bandwidth.

Final design. Logistic regression with hour-of-week and CO_2 features: it is within about 0.01 AUC of the best model, fits about four times faster than HGB, and is interpretable.

7. Limitations and lessons learned

- One working sensor, about 105 days, one zone, one end-of-semester test window. Generalisation to other sensors, rooms or seasons is untested.
- The mapping is inferred. The mapping is inferred, so the results apply to ‘in or near this zone’, not a confirmed room.
- Ground truth is another sensor's count with unknown accuracy; public holidays and semester breaks are not modelled. The control sensor also shows occasional CO_2 spikes of up to about 900 ppm above its weekly profile (122 bins fall outside the range drawn in Figure 5C) that were not removed, so its small loss in the control may partly reflect them.
- Results depend on data volume: the 44-day experiment showed no environmental gain.
- The schedule baseline was stronger in 2025 to 2026 (AUC 0.92 on the control) than in 2024 (0.83); I suspect environmental sensing helps most when the schedule is less predictable, but this was not tested.
- Lessons: audit before modelling; a control that cannot succeed is as informative as one that does; select the mapping and the model on data the test set never saw; and measure before claiming an optimisation works.

8. AI use and reflection

Tool. Claude (Anthropic) in the claude.ai chat interface sonet 5.5 (medium reasoning effort).

What it was used for. Scoping project options; drafting nearly all of the source code (profiling, cleaning, mapping diagnostics, modelling, command line interface, tests, benchmark, figures); designing the validation (residual correlation, placebo test, day-block bootstrap, negative control); debugging terminal and Git problems; and drafting this report from my results.

Approximate contribution. The AI drafted the large majority of the code and much of the analysis design, while I supplied the brief and data, chose the research question, ran everything, supplied every output and made the scope decisions. For the report, AI produced a first draft, which I then edited and used to finalise the findings.

How AI output was checked. A 55-test pytest suite on synthetic data with known answers, including a hand-calculated check of the time weighting and a check that features use only past values; equality checks between baseline and improved code on the real data; a negative-control sensor; reproducing earlier results to three

decimals after refactoring; measuring speedups instead of assuming them; and inspecting generated figures and documentation.

Table 6. Errors or unhelpful AI output, how each was found and the fix.

Error or unhelpful AI output	How it was found	Fix
First variable statistics were skewed: the parser stopped at the first valueless formaldehyde reading, so pressure, temperature and PM statistics used 37% of rows	Second profiling run showed 2,267 valueless formaldehyde entries	Tolerant parser
First mapping screen compared spaces over unequal time windows	Spotted when reading the table	Fair common window diagnostics
Zone choice used a window overlapping the test period (selection leakage)	Raised while reviewing the validation	Redone with training period data only
Documentation called the baseline duplicate counter slow	Benchmark showed 1.1x time difference	Claim corrected to memory
Unstable sort could keep the wrong duplicate	A unit test written during refactoring	Stable sort
A file was never attached to a reply	I noticed it was missing and asked	File supplied
HTML docs rendered badly (reST roles, indented lists)	Screenshot of the docs	Docstrings reformatted
Example paths and instructions caused repeated terminal confusion	Errors reported back	Instructions rewritten

Decisions that remained mine. The research question; treating mapping gap as part of the problem; limiting scope to one sensor and a zone; accepting the two-space zone; threshold used; what to claim and not claim; which results to report.

Effect on my understanding and workflow. AI was highly beneficial as it enabled me to work through the coding problem with my own prompts and ideas while letting the AI focus on the majority of the work. Oftentimes I would have to keep the AI in check/limit its focus or change the direction it was heading.